

Especificacion de Desarrollo PWA

PROMPT DEFINITIVO PARA LEOFIT (SIN EMOJIS)

text

Genera una aplicación web completa tipo PWA (Progressive Web App) con React.js, TailwindCSS y Node.js para gestionar pedidos de una tienda de ropa deportiva llamada "Leofit".

CONTEXTO DEL NEGOCIO

Leofit es una PYME familiar peruana que vende ropa deportiva a través de redes sociales (Facebook y WhatsApp). El dueño, Víctor Raúl Cárdenas Ramírez, actualmente gestiona pedidos de forma manual con cuaderno y teléfono, lo que genera:

- Pérdida de tiempo (horas respondiendo consultas repetitivas)
- Errores en los registros (direcciones equivocadas, productos incorrectos)
- Pérdida de ventas (clientes frustrados por demoras)
- Sin historial de clientes ni análisis de ventas

La solución debe ser una PWA porque:

- Se instala en el celular de Víctor con un solo clic (sin pasar por tiendas de apps)
- Funciona offline (puede ver pedidos guardados en caché)
- Es responsive (mobile-first, porque Víctor usa principalmente su celular)
- Es rápida y liviana
- No requiere publicación en Google Play / App Store

STACK TECNOLÓGICO DEFINIDO

- Frontend: React.js con TailwindCSS (mobile-first, responsive)
- Backend: Node.js con Express (API REST)
- Base de Datos: MongoDB (o MySQL)
- Autenticación: JWT (JSON Web Tokens)
- PWA: Service Worker + Manifest.json
- Despliegue: Vercel (frontend) + Railway (backend)
- Iconos: Google Material Icons (<https://fonts.googleapis.com/icon?family=Material+Icons>) o Font Awesome (gratuito)

PANTALLAS REQUERIDAS (5 pantallas funcionales)

1. Login (Pantalla de Inicio de Sesión)

Propósito: Autenticar al dueño (Víctor) para acceder al sistema.

Elementos:

- Logo de "Leofit" (texto estilizado con tipografía bold, color rojo #E63946)
- Título: "Leofit - Gestión de Pedidos"
- Campo de texto: "Usuario" (placeholder: "correo@leofit.com", type="email", required)
- Campo de texto: "Contraseña" (placeholder: ".....", type="password", required)
- Botón "Iniciar Sesión" (color rojo #E63946, texto blanco, hover: #C62828)
- Enlace pequeño: "¿Olvidaste tu contraseña?" (color gris)
- Diseño centrado vertical y horizontalmente, fondo blanco, sombra suave
- Responsive: en celular, el formulario ocupa el 90% del ancho; en desktop, 400px máximo
- Lógica: Validar que usuario y contraseña no estén vacíos. Simular autenticación con credenciales fijas (usuario: "victor@leofit.com", password: "leofit2026")

2. Dashboard (Panel Principal)

Propósito: Mostrar un resumen rápido del negocio (KPI's y pedidos recientes).

Elementos:

- Barra superior: Logo "Leofit" (izquierda), nombre del usuario "Víctor" (derecha) con icono de persona
- 4 tarjetas resumen en grid (2x2 en celular, 4 en fila en desktop):
- "Pedidos Hoy: 12" (icono de calendario, color azul #1D3557)
- "Pendientes: 5" (icono de reloj, color amarillo #F4A100)
- "En Camino: 3" (icono de camión, color naranja #E67E22)
- "Entregados: 4" (icono de check, color verde #27AE60)
- Tabla "Últimos Pedidos" con columnas: N° Pedido | Cliente | Fecha | Total | Estado
- Estados con colores: Recibido (azul #3498DB), Preparación (amarillo #F1C40F), Camino (naranja #E67E22), Entregado (verde #2ECC71)
- Botón flotante "Nuevo Pedido" (color rojo #E63946, redondeado, con icono de +)
- Lógica: Mostrar datos estáticos de ejemplo (hardcodeados para la demo). Los números de las tarjetas deben ser dinámicos (calcularse a partir de los pedidos).

3. Listado de Pedidos (Historial)

Propósito: Ver, buscar y filtrar todos los pedidos del historial.

Elementos:

- Título: "Historial de Pedidos"
- Barra de búsqueda: "Buscar por cliente..." (input con icono de lupa)
- Filtros en línea:
- Estado: dropdown (Todos, Recibido, Preparación, Camino, Entregado)
- Fecha: input type="date" (desde) y type="date" (hasta)
- Botón "Aplicar Filtros" (pequeño, gris)
- Tabla de pedidos con columnas: N° | Cliente | Fecha | Total | Estado | Acciones
- En "Acciones": botones iconos: "Ver Detalle" (icono de ojo) y "Actualizar Estado" (icono de editar)
- Paginación en la parte inferior: Anterior | 1 | 2 | 3 | ... | Siguiente
- Lógica: Filtrar por estado y fecha (lógica de filtrado en JavaScript). Buscar por cliente. Mostrar 10 pedidos por página.

4. Formulario de Registro de Pedido

Propósito: Registrar un nuevo pedido (simulando atención telefónica/WhatsApp).

Elementos:

- Título: "Registrar Nuevo Pedido"
- Sección "Datos del Cliente" (card con fondo gris claro):
- Campo: "Nombre Completo" (type="text", required, placeholder: "ej. Juan Pérez")
- Campo: "Teléfono" (type="tel", required, placeholder: "ej. 987654321")
- Campo: "Dirección" (type="text", required, placeholder: "ej. Av. Siempre Viva 123")
- Sección "Productos" (card separada):
- Selector: "Producto" (dropdown con opciones: Camiseta Deportiva, Pantalón Jogger, Zapatillas Running, Gorra, Mochila)
- Campo: "Cantidad" (type="number", min="1", default="1")
- Botón "+ Agregar Producto" (color gris, pequeño, con icono de +)
- Tabla de productos agregados con: Producto | Cantidad | Subtotal
- Total a pagar (calculado automáticamente, mostrado en negrita y grande)
- Botones:
- "Guardar Pedido" (verde #27AE60, grande, con icono de guardar)
- "Cancelar" (gris #95A5A6, pequeño)
- Fecha actual: mostrar automáticamente (no editable) en formato "DD/MM/YYYY"
- Lógica: Agregar productos a una lista, calcular total automáticamente. Validar campos obligatorios antes de guardar (mostrar alerta). Guardar el pedido en un array en memoria.

5. Gestión de Productos (Catálogo)

Propósito: Administrar el inventario de productos (CRUD completo).

Elementos:

- Título: "Catálogo de Productos"
- Botón "+ Nuevo Producto" (color rojo #E63946, con icono de +)
- Tabla con columnas: Nombre | Categoría | Talla | Color | Precio (S/) | Stock | Acciones
- En "Acciones": botones "Editar" (icono de editar) y "Eliminar" (icono de basurero)
- Modal (popup) al hacer clic en "+ Nuevo Producto" o "Editar":
- Título: "Agregar Producto" o "Editar Producto"
- Campos:
 - Nombre (text, required)
 - Categoría (dropdown: Deportiva, Casual, Accesorios)
 - Talla (dropdown: S, M, L, XL, Único)
 - Color (input color, o dropdown con colores predefinidos)
 - Precio (number, required, min="0", step="0.01")
 - Stock (number, required, min="0")
- Botones: "Guardar" (verde) y "Cancelar" (gris)
- Confirmación al eliminar: modal con mensaje "¿Eliminar producto X?" y botones "Sí" (rojo) / "No" (gris)
- Lógica: Agregar, editar y eliminar productos de un array en memoria. Mostrar confirmación antes de eliminar.

PALETA DE COLORES (Marca Leofit)

- Primario: #E63946 (rojo) → botones principales, acentos
- Secundario: #1D3557 (azul oscuro) → títulos, encabezados
- Fondo: #F1FAEE (blanco roto) → fondos de pantalla
- Texto: #1A1A1A (negro) → textos principales
- Éxito: #27AE60 (verde) → botón guardar, estado entregado
- Advertencia: #F1C40F (amarillo) → estado preparación
- Info: #3498DB (azul) → estado recibido
- Peligro: #E67E22 (naranja) → estado en camino

ICONOS (Google Material Icons o Font Awesome)

Usar exclusivamente iconos de:

- Google Material Icons: <https://fonts.googleapis.com/icon?family=Material+Icons>
- O Font Awesome (versión gratuita): <https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css>

Iconos requeridos:

- persona (account_circle / fa-user)
- calendario (event / fa-calendar)
- reloj (schedule / fa-clock)
- camión (local_shipping / fa-truck)
- check (check_circle / fa-check-circle)
- lupa (search / fa-search)
- ojo (visibility / fa-eye)
- editar (edit / fa-edit)
- basurero (delete / fa-trash)
- guardar (save / fa-save)
- más (add / fa-plus)
- cerrar (close / fa-times)

ESTRUCTURA DE CARPETAS (Frontend)

src/

- ├── components/
 - | ├── common/
 - | | ├── Button.jsx (botón reutilizable con variantes: primary, secondary, success, danger)
 - | | ├── Input.jsx (campo de texto reutilizable)
 - | | ├── Card.jsx (tarjeta con sombra y padding)
 - | | ├── Badge.jsx (etiqueta de estado con colores)
 - | | └── Table.jsx (tabla reutilizable con columnas dinámicas)
 - | ├── layout/
 - | | ├── Navbar.jsx (barra superior con logo y usuario)
 - | | └── Sidebar.jsx (navegación lateral, opcional)
 - | └── pages/
 - | ├── Login.jsx
 - | ├── Dashboard.jsx
 - | ├── PedidosLista.jsx
 - | ├── PedidoForm.jsx
 - | └── ProductosGestion.jsx
- ├── hooks/
 - | └── useAuth.js (hook para autenticación)
- ├── context/
 - | └── AuthContext.js (contexto para usuario logueado)
- ├── data/
 - | └── mockData.js (datos de ejemplo: productos y pedidos)
- ├── styles/
 - | └── index.css (configuración global de Tailwind)
- ├── App.jsx (routing principal con React Router)
- ├── index.js (punto de entrada, registro de Service Worker)
- ├── manifest.json (configuración PWA)
- └── service-worker.js (Service Worker para offline)

text

ESTRUCTURA DE CARPETAS (Backend - Node.js)

backend/

- ├── src/
 - | ├── controllers/
 - | | ├── authController.js (login, registro)
 - | | ├── pedidoController.js (CRUD de pedidos)
 - | | └── productoController.js (CRUD de productos)
 - | ├── models/
 - | | ├── Pedido.js (modelo MongoDB/Mongoose)
 - | | └── Producto.js (modelo MongoDB/Mongoose)
 - | ├── routes/
 - | | ├── authRoutes.js
 - | | ├── pedidoRoutes.js
 - | | └── productoRoutes.js

```

| | — middleware/
| |   — auth.js (verificar JWT)
|   — config/
|     — database.js (conexión a MongoDB)
— server.js (punto de entrada)
— package.json

```

text

REQUISITOS TÉCNICOS

- Frontend: React.js con TailwindCSS, React Router para navegación, Context API para estado global
- Backend: Node.js con Express, MongoDB con Mongoose (o MySQL con Sequelize), JWT para autenticación
- PWA: Service Worker (cacheo de archivos estáticos), Manifest.json (iconos, tema, nombre)
- Responsive: Mobile-first, usar breakpoints de Tailwind (sm, md, lg, xl). Todos los componentes deben adaptarse a cualquier tamaño de pantalla.
- Accesibilidad: Usar etiquetas <label>, atributos aria-label, roles semánticos
- Código limpio: Nombres de variables descriptivos, comentarios en español
- Iconos: Implementar con Google Material Icons o Font Awesome (sin emojis)

DATOS DE EJEMPLO (Mock Data)

Productos (5 productos)

1. Camiseta Deportiva - Deportiva - M - Rojo - S/49.90 - Stock: 20
2. Pantalón Jogger - Casual - L - Negro - S/79.90 - Stock: 15
3. Zapatillas Running - Deportiva - 42 - Blanco - S/199.90 - Stock: 10
4. Gorra Deportiva - Accesorios - Único - Azul - S/29.90 - Stock: 30
5. Mochila Gym - Accesorios - Único - Negro - S/89.90 - Stock: 8

Pedidos (10 pedidos con diferentes estados)

- 5 pedidos con estado "Recibido"
- 5 pedidos con estado "Entregado"
- (variar fechas, clientes y totales)

ENTREGABLES ESPERADOS

1. Código completo del frontend (React + TailwindCSS + PWA) con las 5 pantallas funcionales
2. Código completo del backend (Node.js + Express + MongoDB) con API REST
3. Configuración de PWA (manifest.json, service-worker.js)
4. Componentes reutilizables (Button, Input, Card, Badge, Table)
5. Navegación entre pantallas con React Router
6. Diseño responsive (mobile-first) con TailwindCSS
7. Datos de ejemplo (mock data) para demostración
8. Archivo README.md con instrucciones de instalación y despliegue

INSTRUCCIONES FINALES

- Genera el código completo y estructurado.
- Incluye comentarios en español para explicar la lógica.
- Asegura que el código sea funcional y ejecutable con npm install y npm start.
- Prioriza la experiencia de usuario: botones grandes, colores consistentes, feedback visual.
- El proyecto debe ser desplegable en Vercel (frontend) y Railway (backend).

- No uses emojis en ningún lugar del código o interfaz. Usa exclusivamente iconos de Google Material Icons o Font Awesome.
- Todos los componentes deben ser 100% responsive (mobile-first).

Genera el código completo para ambas carpetas (frontend y backend) con todas las especificaciones detalladas.